

18. Implement "N-Queen Problem" using Backtracking.

```
#include <stdio.h>

int board [20][20], n;

int isSafe(int r, int c)
{
    int i, j;
    for (i = 0; i < c; i++)
        if (board[r][i])
            return 0;

    for (i = r, j = c; i >= 0 && j >= 0; i--, j--)
        if (board[i][j])
            return 0;

    for (i = r, j = c; i < n && j >= 0; i++, j--)
        if (board[i][j])
            return 0;

    return 1;
}

int solve (int c)
{
    int i;
    if (c == n)
        return 1;

    for (i = 0; i < n; i++)
    {
        if (isSafe(i, c))
        {
            board[i][c] = 1;
            if (solve(c+1))
                return 1;
            board[i][c] = 0;
        }
    }
    return 0;
}
```

```

    }
}
return 0;
}

int main()
{
    int i, j;
    printf("Enter number of queens: ");
    scanf("%d", &n);

    if (solve(0))
    {
        printf("In Solution: \n");
        for (i = 0; i < n; i++) {
            for (j = 0; j < n; j++)
                printf("%d", board[i][j]);
            printf("\n");
        }
    }
    else {
        printf("No Solution Exists");
        return 0;
    }
}

```

o/p:

Enter number of queens: 4
Solution:

0	0	1	0
1	0	0	0
0	0	0	1
0	1	0	0

Perip
25/5/20